

4. Bölüm

C# Programlama Diline Giriş

4.1 Söz Dizim Kuralları

C# dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizimi kullarına** (**syntax**) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.1.1 Kaynak Kodu Oluşturan Karakterler

C# dilinde de kaynak kodu oluşturan karakterler ön tanımlı olarak **UTF-8 karakterleridir**. Ancak talimatları (**statement**) oluşturan anahtar kelimeler (**keyword**) İngiliz alfabesindeki karakterlerden oluşur.

- İngiliz alfabesindeki büyük harfler:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
- İngiliz Alfabesindeki küçük harfler:
a b c d e f g h i j k l m n o p q r s t u v w x y z
- Rakamlar:
0 1 2 3 4 5 6 7 8 9
- Özel Karakterler:
< > . , _ () ; \$: % [] # ? ' & { } " ^ ! * / | - ~ +
- Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (**white space**) karakterleri;
 - Boşluk (**space**)
 - Yeni satır (**new line**)
 - Satır başı (**carriage return**)
 - Geri (**backspace**)
 - Sekme (**tab**)

§4.1.2 Anahtar Kelimeler

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C# derleyicisi tarafından özel anlama sahip önceden tanımlanmış **ayrılmış kelimelerdir** (**reserved word**). Bu kelimeler, C# söz diziminin bir parçası olan önceden tanımlanmış sözcüklerdir ve talimatların (**statement**), bir parçasıdırlar. Başka hiçbir amaç için veya kimliklendirmede kullanılamazlar. Aşağıda anahtar kelimelerin listesi verilmiştir ¹;

Veri Tipleri	Erişim Belirleyiciler	Kontrol Yapıları	Diğerleri
bool	public	if	base
byte	private	else	this
char	protected	switch	new
decimal	internal	case	return
double	static	for	try
float	readonly	foreach	catch
int	virtual	while	finally
long	override	break	throw
object	abstract	continue	using
string	sealed	yield	lock

Tablo 4.1: Temel Anahtar Kelimeler

C# dilinde anahtar kelime olmayıp bağlam (**context**) olarak kullanılan başka anahtar kelimeler de vardır. **Bağlam anahtar kelimeleri** (**contextual keywords**) olarak adlandırılan bu kelimeler, derleyici için sadece belirli kod blokları içinde özel bir anlam taşırlar. Bu sayede, bu kelimeleri değişken kimliği (**identifier**) olarak kullanmanıza teknik olarak izin verilir, ancak kod okunabilirliği açısından bu pek önerilmez.

¹<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/>

Talimatlar ve Anahtar Kelimeler

- ◊ Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**.
- ◊ C# dilinde **kod bloğu** (**code block**) oldukça fazla kullanır. Süslü parantez açtıktan (**{**) sonra kapanana (**}**) kadar yazılanlar **kod bloğu** (**code block**) olarak adlandırılır. **Kod bloklarının sonrasında noktalı virgül kullanılmaz!**
- ◊ **Talimatlar**, yapılaması gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**) ve bu blokların içinde yer alırlar. Talimatlar ve **noktalı virgül karakteri** (**;**) ile biter.
- ◊ Bu kurallara uyulması kaydıyla her türlü boşluk, yeni satır ve sekme gibi beyaz boşluk (**white space**) karakterleri istenildiği gibi kullanılabilir. Bundan dolayı kodlama **gelişigüzel** (**free format**) yapılabilir!

Bağlam olarak kullanılan anahtar kelimelerin çeşitli üstünlükleri vardır;

- ◊ **Esneklik**: Normal anahtar kelimeler (örneğin **class** veya **int**) asla değişken kimliği olamazken, bağlamsal anahtar kelimeleri değişken kimliği yapabilirsin. Örneğin: **var get = 5;** geçerli bir koddur.
- ◊ **LINQ Baskınlığı**: Bağlamsal kelimelerin en yoğun kullanıldığı yer LINQ (**Language Integrated Query**) ifadeleridir. Buradaki **from**, **select** gibi kelimeler sadece sorgu içinde özel anlam taşır. İleride 15.1 başlığında anlatılacaktır.
- ◊ **Modern C# Etkisi**: **init**, **record**, **nint** ve **with** gibi kelimeler C#9.0+ sürümlerinde eklenmiştir.

Kategori	Anahtar Kelimeler
Özellik (property) & İndisleyici (indexer)	get, set, init, value
LINQ Sorguları	select, from, where, join, group, orderby, ascending, descending, into, on, let, equals
Zaman Uyumsuz (asenكرون) Programlama	await, async, yield
Veri Modelleme	record, partial, var, dynamic, nint, nuint
Olay ve Kapsam	add, remove, global, when, managed, unmanaged
Desen Seçme (pattern matching)	not, and, or, with

Tablo 4.2: Bağlamsal Anahtar Kelimeler

Talimatlar ve Ortak Ara Dil Kodu

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, işlemciye gönderilen birden fazla emir kodundan (**instruction code**) oluşur. C# uygulaması, .Net platformu için derlendiğinden işlemcinin emir kodları yerine CIL kodunu kullanır.

§4.1.3 Açıklamalar

C# programlama dilinde kod blokları (**code block**) kullanılarak yazılan kodlar birbirinden ayrılır. İstenildiği kadar iç içe blok yazılabilir. Ayrıca programcı kodu yazarken, **açıklama** (**comment**) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü karakteri (**//**) kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama **/*** ile ***/** karakterleri arasına alınır.

Aşağıda açıklama eklenmiş kod örneği bulunmaktadır.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Merhaba!"); //Talimat ; ile biter.
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C# programıdır.
İlhan ÖZKAN, Ocak 2025
*/
{ // Bi Kod Bloğu Başlangıcı
    { // İç Kod Bloğu Başlangıcı
        //iç bloğa ait girinti...
        Console.ReadLine(); //Konsoldan Satır Okuma Talimatı.
        //...
    } //İç Kod Bloğu Bitişi
} // Bir Kod Bloğu Bitişi
```

C# dilinde kodun bir başka programcı tarafından anlaşılması için bir kod bloğu içindeki açıklama ve talimatları bloktan itibaren girintili olarak yazarız. Kodu bu şekilde yazmamızın derleyici açısından hiçbir önemi yoktur. Derlemenin ilk aşamasında bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.

4.2 Değişken Tanımlama

Değişkenler veri yapılarının temel öğeleridir. Değişkenin (**variable**) ne olduğu 2.4 başlığı altında verilmiştir. Değişkenler, belleğin belli bir bölgesinde yer kaplar ve çeşitli biçimde verilere ilişkin değerleri tutarlar.

Değişkenler tanımlanırken hangi veri tipinden (**data type**) olduğu belirtilir. Bu aynı zaman bellekte ayrılacak yer miktarını ve tutacağı verinin ikili biçimini belirler.

§4.2.1 Değer Tipleri

Değer tipleri (**value type**), veriyi doğrudan yığın bellek (**stack segment**) bölgesinde tutan veri tipleridir. Bir değer tipi değişkeni oluşturduğunuzda, **değişkenin kendisi verinin gerçek değeri olarak hareket eder**.

Değer Tiplerin En Temel Özellikleri

- ◇ **Doğrudan Erişim:** Değişkene eriştiğinizde **doğrudan değere ulaşırsınız**.
- ◇ **Kopyalama Davranışı:** Bir değer tipi değişkenini başka bir değişkene atadığınızda, değer bir kopyası oluşturulur. İki değişken birbirinden bağımsız hale gelir.
- ◇ **Bellek Yönetimi:** Yığın (**stack**) üzerinde tutuldukları için oldukça hızlıdır ve **kapsamdan (scope) çıktıkları anda bellekten otomatik olarak silinirler**.
- ◇ **Boş Olamaz:** Normal şartlarda **değer tiplerin mutlaka bir değeri vardır**. Değeri yok yani **boş (null)** denilemez! Ancak ileride anlatılacak **Nullable<T>** ile değer tipler için bu mümkün hale gelir.

Tamsayı Değer Tipleri

Tip	Boyut (Bit)	.NET Tipi	Değer Aralığı
İşaretili Tam Sayılar			
<code>sbyte</code>	8	<code>SByte</code>	-128 ile 127
<code>short</code>	16	<code>Int16</code>	-32.768 ile 32.767
<code>int</code>	32	<code>Int32</code>	-2.147.483.648 ile 2.147.483.647
<code>long</code>	64	<code>Int64</code>	-9.223.372.036.854.775.808 ile 9.223.372.036.854.775.807
İşaretsiz Tam Sayılar			
<code>byte</code>	8	<code>Byte</code>	0 ile 255
<code>ushort</code>	16	<code>UInt16</code>	0 ile 65.535
<code>uint</code>	32	<code>UInt32</code>	0 ile 4.294.967.295
<code>ulong</code>	64	<code>UInt64</code>	0 ile 18.446.744.073.709.551.615
Platform Bağımlı Tam Sayılar			
<code>nint</code>	32/64	<code>IntPtr</code>	Platforma göre değişir
<code>nuint</code>	32/64	<code>UIntPtr</code>	Platforma göre değişir

Tablo 4.3: Tam Sayı Veri Tipleri ve Özellikleri

Bir tam sayı veri tipinin, hem pozitif hem de negatif değerleri tutabileceğini düşünüyorsak **işaretili (signed)** veri tiplerinden birini seçmeliyiz. Örneğin 8 bitlik `sbyte` veri tipinde en anlamlı olan 8. bitini, işaret biti olarak kullanılmasını sağlar. Böylece -128 ile +127 arasındaki sayılar değer olarak kullanılabilir.

Benzer şekilde bir tam sayı veri tipinin, sadece pozitif değerleri tutabileceğini düşünüyorsak **işaretsiz (unsigned)** bir veri tipini seçmeliyiz. Örneğin 8 bitlik `byte` veri tipinde en anlamlı olan 8. bitinin de sayı olarak kullanılmasını sağlar. Böylece 0 ile 255 arasındaki sayılar değer olarak kullanılabilir.

C# 9.0+ ile gelen platform bağımlı tam sayılar, aslında bellek adreslerini değer olarak tutarlar. Dolayısıyla işlemcinin desteklediği adres genişliğine göre bellek boyutu değişir.

Kayan Noktalı Sayı Değer Tipleri

Kayan noktalı (floating-point) sayı tipleri, hassasiyet ve kullanım amaçlarına göre üç ana kategoriye ayrılır. Burada **decimal** veri tipinin kullanımı finansal hesaplamalar için çok kritiktir.

Diğer Değer Tipleri

Tabloda;

- ◇ Mantıksal olarak **true** ya da **false** tutabilen **bool** değer tipi, şart ifadelerinde (**expression**) kullanılır.

Tip	Boyut	Hassasiyet	.NET Tipi	Sonsuzluk/NaN
<code>float</code>	32 Bit	6-9 basamak	<code>Single</code>	Destekler
<code>double</code>	64 Bit	15-17 basamak	<code>Double</code>	Destekler
<code>decimal</code>	128 Bit	28-29 basamak	<code>Decimal</code>	Desteklemez

Tablo 4.4: Kayan Noktalı ve Ondalıklı Veri Tipleri

Tip	Boyut	.NET Tipi	Açıklama
<code>bool</code>	8 Bit (1 Byte)	<code>Boolean</code>	<code>true</code> veya <code>false</code> değerlerini alır.
<code>char</code>	16 Bit (2 Byte)	<code>Char</code>	Tek bir Unicode karakteri tutar.
<code>object</code>	32/64 Bit	<code>Object</code>	Tüm tiplerin türediği kök tiptir.
<code>dynamic</code>	32/64 Bit	<code>Object</code>	Çalışma zamanında tipi belirlenen değişken.
<code>string</code>	Değişken	<code>String</code>	Karakter dizilerini (metin) tutar.
<code>struct</code>	Değişken	Kullanıcı Tanımlı	Değer tiplerden oluşan yapılar
<code>enum</code>	<code>int</code> kadar	Kullanıcı Tanımlı	İsmlendirilmiş tam sayılar

Tablo 4.5: Mantıksal, Karakter ve Temel Referans Tipleri

- Unicode UTF-16 karakterini tutan 16 bitlik `char` değer tipi. Karakterler `'\u0000'` ile `'\uFFFF'` arasında değer alır. Değeri sıfır olan `'\u0000'` karakterine yani `'\0'` karakterine boş karakter (**null character**) olarak adlandırılır.
- İleride yeri geldikçe göreceğimiz yapı (`struct`) ve numaralandırma (`enum`) tipleri de değer tipleridir.
- İleride göreceğimiz `tuple` bir değer tipidir, ancak basit bir veri tipi değildir.

Değer Tiplerin Bellek Erişim Hızı Yüksek

Değer tipleri doğrudan yığın bellek (**stack**) üzerinde yaşadığı için, bu verilere erişim ve silme işlemleri mikro saniyeler düzeyinde gerçekleşir. **Değer tiplerde bellek erişim hızı çok yüksektir.** Bu, C#'ın neden bu kadar yüksek performanslı bir dil olduğunun temel sebeplerinden biridir.

§4.2.2 Referans Tipler

Referans tipleri (**reference type**), verinin kendisini değil, verinin bellekte bulunduğu adresi (referansı) tutan tiplerdir.

Referans Tiplerin En Temel Özellikleri

- Bellek Yapısı:** Verinin asıl içeriği öbek bellek (**heap**) bölgesinde saklanır; bu verinin adresi ise yığın bellek (**stack**) bölgesindeki değişkende tutulur.
- Kopyalama Davranışı:** Bir referans tipi değişkenini başka bir değişkene atadığınızda, verinin kendisi kopyalanmaz. Sadece adresi kopyalanır. Sonuç olarak iki değişken de bellekteki aynı nesneyi gösterir.
- Varsayılan Değer:** Olmayan nesneleri gösteren referans değişkenler varsayılan olarak boş referanstır (**null reference**). Bu nedenle referans tiplerinin varsayılan değeri `null`'dır.
- Yaşam Döngüsü:** Öbek bellek (**heap**) üzerindeki nesneler, artık hiçbir değişken tarafından referans gösterilmediklerinde **çöp toplayıcı GC** (**garbage collector**) tarafından temizlenir.

Bu tipler ilerleyen bölümlerde yeri geldikçe detaylı olarak anlatılacaktır.

Referans Tipi	Açıklama
<code>class</code>	Kullanıcı tanımlı sınıflar (Örnek olarak <code>string</code> , <code>Form</code> , <code>List</code>).
<code>interface</code>	Sınıfların uygulaması gereken sözleşmeleri tanımlar.
<code>delegate</code>	Yöntem referanslarını (göstericilerini) tutan tiplerdir.
<code>record</code>	Veri odaklı nesneler için kullanılır (C# 9.0+).
<code>dynamic</code>	Tip kontrolünün çalışma zamanında yapıldığı özel referans tipi.
<code>object</code>	Referans tipler de dahil tüm tiplerin en tepedeki kök sınıfıdır.
<code>T[], T[,]</code> ve <code>T[][]</code>	Dizi tipleri.

Tablo 4.6: Referans Tipler ve Özellikleri

Referans Tiplere Bellek Tahsisi

Bir referans tipi öbek (**heap**) bellekte yer tahsisi ancak çalışma zamanında (**run-time**) oluşturabiliriz. Bellek tahsisi için **new işleci** (**new operator**) kullanılır. Ardından varsayılan durumlar için **yapıcılar** (**constructor**) çalıştırılır. Bu tiplerin yapıcıları, öbek bellekte yer ayrıldıktan sonra nesnenin durumlarını yani verilerini düzenler ve istenilen verilerle imal edilmelerini sağlar.

§4.2.3 Değişken Bildirimi ve Tanımlaması

Bildirim (**declaration**), derleyiciye bir değişkenin adını ve veri tipini tanıtmaktır. Değişken bildiriminde, eğer tipleri için yığın bellekte (**stack**) yer tahsis edilir, ancak referans tipleri için sadece bir gösterici (**pointer**) oluşturulur ve değeri **null** olur. Değişken bildirimine iki örnek verelim;

- ◆ Değer Tip Örneği: örnek olarak **int sayi**; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim **sayi** adında bir değişkenim olacak ve bu **int** tipinde veri tutacak."
- ◆ Referans Tip Örneği: örnek olarak **Oğrenci ali**; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim **Oğrenci** sınıfından imal edilecek bir nesneye **ali** adında referansım olacak ve **ali** henüz bir **Oğrenci** tipinde bir nesneyi göstermiyor. Çünkü değeri **null**"

Değişkenlerin veri tipine (**data type**) bağlı olarak benzersiz bir **kimlik** (**identifier**) verilerek **bildirilmesi** (**declaration**) gerekir!

```
byte yas; // veri tipi "char" ve kimliği "yas" olan değişken bildirimi
int kat; // veri tipi "int" ve kimliği "kat" olan değişken bildirimi
float kilo; // veri tipi "float" ve kimliği "kilo" olan değişken bildirimi
decimal para; // veri tipi "decimal" ve kimliği "para" olan değişken bildirimi
```

Tanımlama (**definition**), bildirimi yapılmış bir değişkene ilk değerinin atanması veya (referans tipleri için) bellekte gerçek bir nesnenin oluşturulması sürecidir. Değer tipe değer atanır veya referans tipe öbek bellek (**heap**) bölgesinde nesne için yer tahsis edilir. Referans tipe bellekte yer tahsis edildikten sonra varsayılan durumlar için **yapıcılar çalıştırılır**. Değişken tanımlamaya, **başlatma** (**initialization**) adı vermek yaygın bir kullanımdır. Değişken tanımlamasına iki örnek verelim;

- ◆ Değer Tip Örneği: örnek olarak **int sayi=10**; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "Benim **sayi** adında bir değişkenim olacak ve bu **int** veri tipinde 10 verisini tutacak."
- ◆ Referans Tip Örneği: örnek olarak **Oğrenci ali = new Oğrenci()**; talimatı (statement) yazdığımızda derleyiciye şu mesajı veririz: "**Oğrenci** sınıfından bir nesne imal et ve **ali** adında referansım onu göstersin."

Değişkenleri kullanabilmek için bildirim yapılması yeterlidir. Ancak ilk değer verilerek de değişkenler başlatılabilir. Böylece bildirim de yapılmış olunur. Şimdi yukarıda bildirimi yapılan değişkenleri başlatan bir örneği yenileyebiliriz.

```
byte yas=45; // veri tipi "char" ve kimliği "yas" olan değişken 45 değeriyle başlatıldı
int kat=1; // veri tipi "int" ve kimliği "kat" olan değişken 1 değeriyle başlatıldı
float kilo=60.0F; // veri tipi "float" ve kimliği "kilo" olan değişken 60.0 değeriyle başlatıldı
decimal para=200m; // veri tipi "decimal" ve kimliği "para" olan değişken 200 değeriyle başlatıldı
```

Bildirim

Bildirim (**declaration**) derleyiciye böyle bir değişken veya nesne var demektir. Bunu ileride daha detaylı göreceğiz. Şimdilik tanımlama gibi kabul edebilirsiniz. **Tanımlama**, (**definition**) derlemenin **bağlama** (**link**) ile birlikte sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

§4.2.4 Değişken Kapsamı

Değişkenin kapsamı (**variable scope**); değişkenin bildiriminin (**variable declaration**) yapılabileceği, değişkene erişilebileceği (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. Değişkenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

4.2.3 Başlığında bildirimi yapılan **yas**, **kat**, **kilo** ve **para** değişkenleri ana fonksiyon olan **Program.cs** içinde tanımlandığından, tanımlanan değişkenler sadece program içinde geçerlidir. Yani değişkenin kapsamı (**scope**), bildirimi yapıldıktan sonra dosya sonuna kadardır. **Aynı kapsam içinde bildirilmiş bir değişken kimliği, bir başka değişkene verilemez!**

Değer Tiplerin En Temel Özellikleri

Süslü parantez ile tanımlanan **her kod bloğu (code block)** içinde de **değişken bildirimi yapılabilir**. Bu şekilde tanımlanan değişkenin kapsamı, **kod bloğu ile sınırlıdır**. Yani bildirimi yapıldıktan sonra değişken blok sonuna kadar geçerlidir.

Yöntemlerin de en az bir kod bloğu vardır. Bu kod blokları içinde tanımlanan değişkenlere **yerel değişken (local variable)** adı verilir. Yerel değişkenlerin kapsamı da fonksiyon bloğu ile sınırlıdır.

§4.2.5 Değişken Kimliklendirme Kuralları

C# programlama dilinde **değişkenlerin kimliklendirilmesinde (identifier definition)** dört temel kural vardır;

Değişken Kimliklendirme Kuralları

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. Kimliklendirmede yukarıdaki iki kurala uyacak şekilde herhangi bir karakter kullanılabilir.
4. Aynı kapsam (**scope**) içinde **aynı kimlik iki farklı değişkene verilemez!**

Aşağıda kimliklendirme örnekleri verilmiştir;

```
//Tamsayı Değişken Bildirimleri:
byte yaş;
int tamsayı;
long uzunTamsayı;
//Gerçek Sayı Değişken Bildirimleri:
float oran;
double gerceksayı;
/*Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
int kat1, kat2, kat3;
float ölçek1, ölçek2;
decimal alınanÜcret, paraÜstü;
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığı artırmak için **Deve Yazımı (camel case)** şeklinde olarak kimlik verilir. Bu kimliklendirme türünde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir. Aşağıdaki buna ilişkin bir örnek verilmiştir.

```
int sayaç;
int öğrenciBoy=180;
int asansörünOlduğuKat=-1;
float asansörAğırlığı=300.0F;
byte öğrenciYaşı;
char ilkHarf='A';
double ortalamaNot=0.0;
```

4.3 Değişmezler

Programcı, kodlama yaparken bazı **değişmezleri (literal)** ister istemez kullanır. Bu değişmezler, derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısıdır. Değişkenlere verilen **ilk değerler (initial value)** ile **katsayılar (coefficient)** en çok kullanılan değişmezlerdir. Örnek olarak `int sayi = 10;` talimatında `sayi` bir değişkendir, `10` ise bir tam sayı değişmezidir (**integer literal**).

Değişkenlerin veri tiplerine göre değişmezler çeşitli biçimlerde yazılabilir.

Kod İçerisinde Bindelik ve Ondalık Ayracı

Türkçe **1.025,75** olarak yazılan bir gerçek sayı, İngilizce **1,025.75** olarak yazılır. Türkçede bindelik ayracı nokta ve ondalık ayracı virgül iken, İngilizcede bindelik ayracı virgül, ondalık ayracı noktadır. Kodlama İngilizce olarak yapıldığından **gerçek sayılar İngilizcedeki gibi yazılır**.

Aşağıda bazı değişmezlerle ilişkin örnek verilmiştir;

Veri Tipi	Değişmez Örneği	Notlar / Sonekler
Tam Sayılar		
<code>int</code>	<code>100, 0b110, 0x1A</code>	Varsayılan tam sayı tipidir.
<code>uint</code>	<code>100u, 100U</code>	<code>u</code> veya <code>U</code> soneki alır.
<code>long</code>	<code>100l, 100L</code>	<code>l</code> veya <code>L</code> soneki alır (<code>L</code> önerilir).
<code>ulong</code>	<code>100ul, 100UL</code>	<code>ul</code> , <code>uL</code> , <code>Ul</code> veya <code>UL</code> olabilir.
<code>byte / sbyte</code>	<code>(byte)10, (sbyte)-5</code>	Doğrudan soneki yoktur, cast edilir.
<code>short / ushort</code>	<code>(short)500</code>	Doğrudan soneki yoktur, cast edilir.
Kayan Noktalı ve Ondalıklı Sayılar		
<code>double</code>	<code>1.5, 1.5d, 1e2</code>	Varsayılan ondalıklı tiptir.
<code>float</code>	<code>1.5f, 1.5F</code>	<code>f</code> veya <code>F</code> soneki zorunludur.
<code>decimal</code>	<code>1.5m, 1.5M</code>	<code>m</code> veya <code>M</code> soneki zorunludur.
Diğer Değer Tipleri		
<code>bool</code>	<code>true, false</code>	Sadece bu iki anahtar kelime.
<code>char</code>	<code>'A', '\n', '\x0041'</code>	Tek tırnak ve kaçış dizileri kullanılır.
<code>enum</code>	<code>Gunler.Pazartesi</code>	Tanımlanan enum üyesi kullanılır.

Tablo 4.7: Değer Tipler İçin Değişmez Biçimleri

```

uint işaretliTamsayı = 5U;
int tamsayı = -5;
sbyte işaretliBayt = -120;
decimal ondalık = 30.5M;
double çiftİncelikliKayanNoktalıSayı = 30.5D;
float kayanNoktalıSayı = 30.5F;
long uzunTamsayı = 5L;
ulong işaretliUzunTamsayı = 5UL;
byte bayt = 127;
short kısaTamsayı = -12;
ushort işaretliKısaTamsayı = 220;
bool evetHayır = true;
bool doğruYanlış = false;
char karakter1 = 'ö'; // Karakter değişmezleri tek tırnak içinde yazılır.
char karakter2 = '\u006A'; // Tek tırnak içinde UTF-16 kodlamasına göre
char karakter3 = '\x0013';
char karakter4 = 'Ş';
/*
Metin değişmezler çift tırnak arasında yazılır.
Metin içinde;
Çift tırnak kullanmak için \" yazılır.
Bir sonraki satıra geçmek için \ karakteri kullanılır.
\ karakteri kullanmak için \\ yazılır.
*/
string metin = "hello, this is a string literal";
/*
Yukarıdaki metin değişmesinde birçok kural vardır.
Onları gözardı etmek için @ karakteri ile başlayan tam metin (verbatim string) değişmesi
↪ kullanılır.
Tam metinlerde sadece çift tırnak için iki adet "" kullanılır.
*/
string tammetin = @"İlhan
Özkan \ """;

```

Aşağıdaki kod örneğinde programcı, `3F`, `3.14F`, `3.14F` ve `2.0F` olmak üzere dört farklı değişmez kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir.

```

float yariCap = 3F;
float daireninAlani = 3.14F * yariCap * yariCap;
float daireninCevresi = 2.0F * 3.14F * yariCap;

```

Boş Karakter Değişmezi

Kodu 0 olan karakter ise **boş karakter** (**null character**) olarak adlandırılır.

4.4 Yerel Sabitler

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir montaj kodu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye yerel değişkenler **sabitler** (**const**) olarak tanımlanabilir. Bunun için **const** anahtar kelimesi sıfat olarak kullanılır ve bu değişken **yerel sabit** adını alır. Başlatması yapıldıktan sonra yerel sabitler ve değiştirilemezler. Sabit değişkenler etik olarak büyük harfle kimliklendirilir.

```
const float PI=3.14F; //Yerel sabiti başlatma yapmazsak daha sonra değiştiremeyiz!
float yariCap = 3F;
float daireninAlani = PI * yariCap * yariCap;
float daireninCevresi = 2.0F * PI * yariCap;
```

Örnekteki aynı yerel sabiti PI birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Bildirim sırasında **const** sıfatı kullanılan yerel sabit için bildirim yapılmamalı değişken başlatılmalıdır. Bir sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- ♦ **Gelişmiş Kod Okunabilirliği:** Bir değişkene **const** sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- ♦ **Gelişmiş Veri Tipi Güvenliği:** **const** kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- ♦ **Gelişmiş Optimizasyon:** Derleyiciler, değerlerinin programın icrası sırasında değişmeyeceğini bildikleri için yerel sabitleri daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- ♦ **Daha İyi Bellek Kullanımı:** Değişkenlere **const** sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- ♦ **Gelişmiş Uyumluluk:** Değişkenlere **const** sıfatı vermek, kodunuzdaki yerel sabitleri kullanan diğer kodlar ve API'lerle daha uyumlu hale getirebilir.
- ♦ **Gelişmiş Güvenilirlik:** **const** kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.5 Anonim Tipler

Değişkenler veri tipi verilmeden de tanımlanabilir. Bu durumda veri tipi değişkene verilen ilk değer değişmezinden (**literal**) çıkarılır. Bu tip değişkenlere **anonim tip** (**anonymous type**) adı verilir. Anonim tipler **var** anahtar kelimesiyle tanımlanır ve **statik tip denetimini korur** yani tip denetimi derleme zamanı (**compile-time**) yapılır. Aşağıda değer tipler için bir örnek verilmiştir;

```
var pi=3.14F; //pi:float
var sayac=0; //sayac:int
var durum=false; //durum:bool
var gercek=10.5; //gercek:double
var ondalıklı=100000.0m; //ondalıklı:decimal
```

Aşağıda verilen örneklerdeki **anonim tipler** (**anonymous type**) aslında referans tiptir. Referans tipler ileride detaylı anlatılacaktır. Örnek olarak **var anon=new { x = 1, y = 2 }** talimatını göz önüne alalım;

- ♦ Önceden bir **class** veya **struct** tanımlaması yapmadan, sadece o anlık ihtiyacımızı karşılayacak geçici bir nesne öbek bellekte (**heap**) oluşturuyoruz. Oluşturulan nesne **isimsizdir**. Yazdığımız kod tarafında bu tipe verebileceğiniz bir isim yoktur. Bu yüzden **var** kullanmak zorunludur.
- ♦ Oluşturulan nesne salt okunurdur (**read-only**). Oluşturulan bu nesnenin özellikleri de (x ve y) daha sonra değiştirilemez. Yani **anon.x = 5;** yazarsanız derleme hatası alırsınız.
- ♦ Anonim tipler arka planda birer **class** olarak oluşturulur, dolayısıyla belleğin öbek bellek (**heap**) bölgesinde yaşarlar.

```
var anon = new { Value = 1 };
Console.WriteLine(anon.Value); //1
//Console.WriteLine(anon.Id); Derleme Hatası Olur!
var anon2 = new { x = 1, y = 2 };
// anon.x == 1
// anon.y == 2
int a = 10;
```



```
int b = 20;
var anon3 = new { a, b };
// anon3.x == 10
// anon3.y == 20
var anon4 = new { adı="ilhan", yaşı=50 }; //Anonim Tipler ilerde anlatılacaktır.
// anon4.adı == "ilhan"
// anon4.yaşı == 50
```

Anonim tiplerin eşitliği de sorgulanabilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 1, Bar = 2 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon4 = new { Bar = 2, Foo = 1 };
// anon.Equals(anon2) == true
// anon.Equals(anon3) == false
// anon.Equals(anon4) == false (anon ve anon4 farklı tiplerdir)
```

Anonim tiplerin içerdiği özellikler aynı ise aynı olarak kabul edilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 7, Bar = 1 };
var anon3 = new { Bar = 1, Foo = 3 };
var anon4 = new { Fa = 1, Bar = 2 };
// anon ve anon2 aynı tiptir
// anon ve anon3 farklı tiptir (Bar ve Foo farklı sırada olduğundan)
// anon ve anon4 farklı tiplerdir (özellik isimleri farklı olduğundan)
```

4.6 Dinamik Değişkenler

Dinamik değişkenler (**dynamic variables**), anonim değişkenlerin aksine, değişkenlerin tipi dinamik olarak derleme zamanı yerine çalışma zamanında (**run-time**) veri tipi kontrolü yapılan değişken tipidir.

dynamic anahtar kelimesi, dilin statik yani derleme zamanı (**compile-time**) yapısından dinamik yani çalışma zamanı (**run-time**) yapısına geçtiği sihirli bir köprüdür. C#4.0+ ile gelen bu özellik, derleyiciye şu mesajı verir: "Bu değişkenle ne yapacağıma şimdi karışma, çalışma zamanında (**run-time**) bakarız."

```
dynamic val = "Merhaba";
Console.WriteLine(val.Id); /* Derlenir, ama çalışma anında istisna olur! Çünkü ID adlı bir
↳ özelliği bulunmamaktadır */
```

Normalde C# derleyicisi, bir değişken üzerinde yöntem çağırıldığında o yöntemin var olup olmadığını derleme aşamasında kontrol eder. Ancak **dynamic** olarak tanımlanan bir değişkende bu kontrol atlanır;

- ♦ Yanlış bir yöntem çağırırsanız bile kod derlenir, ancak program çalışırken hata (**RuntimeBinderException**) verir. Bu nedenle **derleme zamanı güvenliği yoktur**.
- ♦ Bir **dynamic** değişken önce **int**, sonra **string** tutabilir. Yani **tip değişebilir!**
- ♦ Çalışma zamanında **object** gibi davranır ama üzerindeki her işlem dinamik olarak çözümlenir.

Aşağıda buna ilişkin bir örnek verilmiştir;

```
dynamic veri = 10;
Console.WriteLine(veri.GetType()); // Çıktı: System.Int32
veri = "Merhaba C#"; //Tip Değişikliği!
Console.WriteLine(veri.GetType()); // Çıktı: System.String
dynamic nesne = "C# Kitabı";
try {
    // string sınıfında 'ToUpper' vardır, sorunsuz çalışır.
    Console.WriteLine(nesne.ToUpper());
    // 'Yuz' diye bir metod yoktur. Derleme hatası ALMAZSINIZ. Çünkü derleyici Yuz() diye bir
    ↳ metodun olup olmadığını kontrol etmez!
    // Ancak çalışma zamanında PROGRAM ÇÖKER.
    nesne.Yuz();
}
catch (Exception ex) {
    Console.WriteLine("Hata: " + ex.Message);
}
```

4.7 Boş Olabilir Değer Tipler

Değer tiplerin alabileceği alt ve üst sınırlar bellidir. Ancak bir değer tipe, değeri yok anlamında **boş** (**null**) atayamazsınız. Bunu sağlamak için değişkeni **boş olabilir** (**nullable**) şekilde aşağıdaki gibi tanımlama yapılır;

```
Nullable<int> i1 = null;
//ya da kısaca veri tipi sonunda ? karakteri kullanılır!
int? i2 = null;
//ya da
var i3 = (int?)null;
```

Boş olabilir (**nullable**) değişkeni aşağıdaki şekilde **null** olmayan bir değerle tanımlayabilirsiniz.

```
//Nullable<int> i = 0;
//Ya da kısaca:
int? i = 0;
```

Boş olabilen değişkenlerde **null denetimi** ve **varsayılan değer atama** işlemlerini tek bir satıra indiren **null birleştirme işleci** (**null coalescing operator**) (**??**) vardır;

```
string isim = GetUser() != null ? GetUser() : "Anonim"; //Eski yöntem.
string isim = GetUser() ?? "Anonim"; //Yeni Yöntem: null coalescing operator
```

Birden fazla **null** ihtimalini ardı ardına kontrol etmek için bu işleci zincirleyebilirsiniz. Bu, "bulabildiğin ilk geçerli değeri al" mantığıdır.

```
string adres = evAdresi ?? isAdresi ?? "Adres Bilgisi Yok";
```

C#8.0+ ile gelen **null atama işleci** (**??=**), bir değişken eğer **null** ise ona bir değer atamak için kullanılır. Eğer değişken zaten bir değere sahipse, atama işlemi gerçekleşmez;

```
List<int> sayilar = null;
// Eger sayilar null ise yeni bir liste olustur ve ata
sayilar ??= new List<int>();
sayilar.Add(1);
```

Null Birleştirme İşleci

Null birleştirme işleci (**??**) sadece kodunuzu kısaltmaz, aynı zamanda **null reference exception** hatalarının önüne geçmek için en güçlü savunma hatlarınızdan biridir!

Ayrıca boş olabilir (**nullable**) bir veri tipini değer tipe aşağıdaki şekilde atayabiliriz.

```
int? i = 10;
int j = i ?? 0; // null ise 0 - null coalescing operator
int j = i.GetValueOrDefault(0); // null ise 0
int j = i.HasValue ? i.Value : 0; // null ise 0 - conditional tenary operator
```

4.8 Numaralandırma Tipi

Numaralandırma (**enumeration**) olarak adlandırılan **enum**, bir grup tam sayı ile ilişkilendirilmiş değişmezden (**literal**) oluşan numaralandırılmış kullanıcı tanımlı bir değer veri tipidir.

Bir **enum** tam sayı veri tiplerinin (**byte**, **sbyte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**) herhangi birinden türetilir. Varsayılan olarak **int** veri tipi altta yatar ve **enum** tanımında veri tipi belirterek değiştirilebilir;

```
enum MedeniHal { Belirtilmemiş, Evli, Bekar, Boşanmış, Dul }; //Ön tanımlı olarak int
enum Günler:byte { Pazartesi = 1, Salı = 2, Çarşamba = 3, Perşembe = 4, Cuma = 5 }
enum Aylar:int { Ocak = 1, Şubat = 2, Mart = 3, Nisan = 4, Mayıs = 5, Haziran = 6, Temmuz = 7,
    ↳ Ağustos = 8, Eylül = 9, Ekim = 10, Kasım = 11, Aralık = 12 }
enum Renkler:uint { Kırmızı = 1, Mavi = 2, Yeşil = 3, Sarı = 4, Beyaz = 5, Siyah = 6 }
```

```
byte işeBaşlananGün=1; // Pazartesi
Günler işbaşı = Günler.Pazartesi;
Aylar ramazan = Aylar.Mart;
Renkler arabanınRengi = Renkler.Kırmızı;
```

Numaralandırmalar bir tam sayının her bir bitini temsil edecek şekilde bayrak (**flag**) olarak kullanılabilir. Burada **int** veri tipinin 32 bitten oluştuğu unutulmamalıdır!

[Flags]

```

enum Bayraklar
{
    //None = 0
    BirinciDurum = 1, //1.Bit
    İkinciDurum = 2, //2.Bit
    ÜçüncüDurum = 4, //3.Bit
    DördüncüDurum = 8, //4.Bit
    BeşinciDurum = 16 //5.Bit
}
[Flags]
enum FlagsEnum
{
    None = 0,
    Option1 = 1,
    Option2 = 2,
    Option3 = 4,
    Default = Option1 | Option3,
    All = Option1 | Option2 | Option3,
}
var İkiBayrakHavada = Bayraklar.BeşinciDurum | Bayraklar.ÜçüncüDurum;
Console.WriteLine(İkiBayrakHavada); //ÜçüncüDurum, BeşinciDurum
var ÜçüncüBayrakHavada = Bayraklar.ÜçüncüDurum;
Console.WriteLine(ÜçüncüBayrakHavada); //ÜçüncüDurum
var BaşkaİkiBayrakHavada = Bayraklar.BirinciDurum | Bayraklar.İkinciDurum;
Console.WriteLine(BaşkaİkiBayrakHavada); //BirinciDurum, İkinciDurum

```

4.9 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı CIL koduna çevrilen programımız aslında yazdığımız talimatları (**statement**) sırasıyla çalıştırır. Aşağıdaki örnek programı göz önüne alalım;

```

1 // See https://aka.ms/new-console-template for more information
2 float boy = 1.80F;
3 float kilo = 100.0F;
4 float bki;
5 float boyKare;
6 boyKare = boy * boy;
7 bki = kilo / boyKare;
8 boy = 1.50F;

```

Yapısal programlamada program, ana fonksiyondan başlar. Yeni proje şablonu ile oluşturulan projelerde **Program.cs** ana fonksiyonu kodladığımız yerdir. Bu nedenle icra, **Program.cs** içindeki ilk talimatla başlar sırayla icra edilir.

1. Satırda, .Net6+ sonrasında yeni proje şablonuyla oluşturulan yeni konsol uygulamasının açıklama satırı bulunur.
2. İcra edilecek ilk talimat 2. satırdaki talimattır. **boy** değişkeni **1.80F** değeri ile başlatılır.
3. Saha sonra 3. satırdaki talimat icra edilir. **kilo** değişkeni **100.0F** değeri ile başlatılır.
4. Ondan sonra 4. satırdaki talimat icra edilir. **bki** değişkeni bildirilir.
5. Daha sonra 5. satırdaki talimat icra edilir. **boyKare** değişkeni bildirilir.
6. Ondan sonraki talimat 6. satırdaki talimattır. **boy** ile **boy** değişkeni çarpılır ve sonuç **boyKare** değişkenine atanır. Artık **boyKare** değişkeninin değeri bu noktadan sonra **3.24** olmuştur.
7. Daha sonra 7. satırdaki talimat icra edilir. **kilo** değişkeni **boyKare** değişkenine bölünür (100,0/3,24) ve sonuç **bki** değişkenine atanır. Artık **bki** değişkeninin değeri bu noktadan sonra **30.86419753** olmuştur.
8. Son olarak 8. satırdaki talimat icra edilir. **boy** değişkenine **1.50** atanmıştır. Bu atama sonrası **boyKare** ve **bki** değişkenlerinin değeri değişmemiştir. Çünkü bu değişkenleri değiştiren 6. ve 7. satırdaki talimatlar icra edilmiştir.

4.10 Temel İşleçler

İşleçler (**operator**) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (**operand**) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de

bulunur.

En Çok Kullanılan Atama İşleci

En çok kullanılan işleç, atama (=) işlecidir. **Atama işleci, sağdaki işleneni soldakine atar.** Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

§4.10.1 Aritmetik İşleçler

C# programlama dilinde **aritmetik işleçlerin** (**arithmetic operator**) çalışma şekli işlenenlerin (**operand**) veri tipine göre değişir. Bu işleçler Tablo 4.8'de verilmiştir. Bu işleçler `int`, `uint`, `long` ve `ulong` tiplerini işlenen olarak alır. İşlenenler diğer tam sayı tipler olduğunda (`sbyte`, `byte`, `short`, `ushort` veya `char`) değerleri bir işlemin sonuç tipi olan `int` tipine dönüştürülür.

İşleç	Adı	Açıklama
+	Toplama	İki işleneni toplar. Sayısal olmayan bazı referans tiplerde (örn. <code>string</code>) birleştirme yapar.
-	Çıkarma	Soldaki işlenenden sağdakini çıkarır.
*	Çarpma	İki işleneni çarpar.
/	Bölme	Soldaki işleneni sağdakine böler. Her iki işlenen tamsayı ise sonuç tamsayıdır (ondalık kısım atılır).
%	Modül	Soldaki işlenenin sağdakine bölümünden kalan değeri verir.

Tablo 4.8: Aritmetik İşleçler

Aritmetik işleçlere (**arithmetic operator**) ilişkin örneği aşağıdaki görebilirsiniz;

```
int a = 9, b = 4, res;

res = a + b; // işlem sonucu res=13
res = a - b; // işlem sonucu res=5
res = a * b; // işlem sonucu res=36
res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
/* Toplama işlecinin işlenenleri tamsayı olduğundan, bölme sonucundaki tam kısım bölme
   ↳ sonucudur.*/
res = (int)(a / 4.3);
/* İşlem sonucu double olacaktır. Bu durumda res değişkeni de int olduğundan sonucu int türüne
   ↳ çevirmemiz gerekir. res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */
res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
```

```
float fa = 6.6F, fb = 2.2F, fres;
fres = fa / fb; // işlem sonucu res=6.6/2.2=3.00
```

Tam sayılarda bölme işlemi, işlemcinin en basit yetenekleriyle çözülür. Aşağıda buna ilişkin örnek verilmiştir.

```
int a = 19 / 2; /* a = 2*9+1 = 9 */
int b = 18 / 2; /* b = 9 */
int c = 255 / 4; /* c = 4*63+3 = 63 */
int d = 45 / 4; /* d = 4*11+1 = 11 */
double aa = 19 / 2.0; /* aa = 9.5 */
double bb = 18.0 / 2; /* bb = 9.0 */
double cc = 255 / 2.0; /* cc = 127.5 */
double dd = 45.0 / 4; /* dd = 11.25 */
double ee = 45 / 4; /* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
```

§4.10.2 Tekli İşleçler

Tekli işleçler (**unary operator**) sadece **bir işlenen üzerinde işlem yapar.** Özellikle bellek yönetimi ve mantıksal akış bölümlerinde sıkça karşımıza çıkar.

Bunların bir kısmı konusu geldikçe kullanımı anlatılacaktır. Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```
int a = 5, b = 5, res;
// 1. Tekli Eksi (Unary Minus)
res = -a; // res = -5
```

İşleç	Adı	Açıklama
+	Tekli Artı	İşlenenin işaretini pozitif yapar (genellikle varsayılan durumdur).
-	Tekli Eksi	İşlenenin işaretini tersine çevirir (x ise $-x$, $-x$ ise x yapar).
++	Arttırım	İşlenenin değerini 1 artırır. Önek (prefix) ise önce artırır sonra değeri döndürür; sonek (postfix) ise önce değeri döndürür sonra artırır.
--	Eksiltme	İşlenenin değerini 1 eksiltir. Önek ve sonek kullanım kuralları artırım işleci ile aynıdır.
!	Mantıksal Değil	bool değerin tersini alır (true ise false yapar).
(T)	Tip Dönüşümü	İşleneni parantez içinde belirtilen veri tipine (casting) dönüştürür.
&	Adres Operatörü	unsafe kod bloklarında değişkenin bellek adresini döndürür.
*	Dolaylı Erişim	unsafe kod bloklarında göstericinin (pointer) işaret ettiği değeri döndürür.
sizeof	Bellek Boyutu	sizeof temel veri tiplerin bellek boyutunu geri döndürür.

Tablo 4.9: Tekli İşleçler

```
// 2. Önce-Arttırım (Pre-Increment)
// a önce 6 olur, sonra 2 ile toplanır.
res = ++a + 2;
/* Sonuç: res = 8, a = 6 */
// 3. Sonra-Arttırım (Post-Increment)
// b'nin mevcut değeri (5) ile 2 toplanır, sonra b 6 olur.
res = 2 + b++;
/* Sonuç: res = 7, b = 6 */

a = 5; b = 5;
// 4. Önce-Eksiltme (Pre-Decrement)
// a önce 4 olur, sonra 2 ile toplanır.
res = --a + 2;
/* Sonuç: res = 6, a = 4 */
// 5. Sonra-Eksiltme (Post-Decrement)
// b'nin mevcut değeri (5) ile 2 toplanır, sonra b 4 olur.
res = 2 + b--;
/* Sonuç: res = 7, b = 4 */

// 6. Mantıksal Değil (!)
// C#'ta ! işleci sadece bool ile çalışır. C dilindeki gibi 0/1 (int) kabul etmez.
bool mantiksalA = true;
bool mantiksalB = false;
bool resBool;
resBool = !mantiksalA; // resBool = false
resBool = !mantiksalB; // resBool = true

// 7. Bellek Boyutu (sizeof)
// sizeof C#'ta genellikle temel veri tipleriyle kullanılır.
int boyutInt = sizeof(int); // sonuç: 4
int boyutChar = sizeof(char); // sonuç: 2 (C#'ta char Unicode'dur!)

// Değişkenler için sizeof ancak 'unsafe' bloklarda veya Marshal.SizeOf yöntemiyle
↪ kullanılabilir. İleride Anlatılacaktır.
```

§4.10.3 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. İlişkisel İşleçler, ileride göreceğimiz karar yapılarının (**if**, **switch**) ve döngülerin (**while**, **for**) temelini oluşturur. Bu işleçler, iki değeri karşılaştırıp sonucu her zaman bir mantıksal değer olan **bool** veri tipinde döndürürler.

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
int a = 5, b = 6;
bool res;
res = a < b; // Küçüktür: a, b'den küçük olduğu için sonuç true
res = a <= b; // Küçük veya Eşit: a, b'den küçük olduğu için sonuç true
```

İşleç	Adı	Açıklama
==	Eşittir	Sol ve sağdaki işlenenler birbirine eşitse true döner.
!=	Eşit Değildir	İşlenenler birbirine eşit değilse true döner.
>	Büyüktür	Soldaki işlenen sağdakinden büyükse true döner.
<	Küçüktür	Soldaki işlenen sağdakinden küçükse true döner.
>=	Büyük veya Eşit	Soldaki işlenen sağdakinden büyük veya ona eşitse true döner.
<=	Küçük veya Eşit	Soldaki işlenen sağdakinden küçük veya ona eşitse true döner.

Tablo 4.10: İlişkisel İşleçler ve Mantıksal Karşılıkları

```
res = a > b; // Büyüktür: a, b'den büyük olmadığı için sonuç false
res = a >= b; // Büyük veya Eşit: a, b'den büyük veya eşit olmadığı için sonuç false
res = a == b; // Eşittir: a ve b değerleri farklı olduğu için sonuç false
res = a != b; // Eşit Değildir: a ve b farklı olduğu için sonuç true
```

§4.10.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operator**) özellikle ikilik tabandaki **tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir**. Aslında bu işlemler, işlemci üzerinde çarpma, toplama, bölme ve çıkarma işlemlerinin temelini oluşturur. Bu işleçler genellikle düşük seviyeli programlama, protokol geliştirme, kriptografi ve performans optimizasyonu gibi alanlarda kullanılır.

İşleç	Adı	Açıklama
&	VE (AND)	Karşılıklı iki bit de 1 ise sonuç 1, diğer durumlarda 0 olur.
	VEYA (OR)	Karşılıklı bitlerden en az biri 1 ise sonuç 1, ikisi de 0 ise 0 olur.
^	ÖZEL VEYA (XOR)	Bitler birbirinden farklıysa 1, aynıysa 0 olur.
~	DEĞİL (NOT)	Tekli işleçtir. Tüm bitleri tersine çevirir (0 → 1, 1 → 0).
<<	Sola Kaydırma	Bitleri belirtilen sayı kadar sola kaydırır. Sağdan 0 ile doldurur.
>>	Sağa Kaydırma	Bitleri belirtilen sayı kadar sağa kaydırır.

Tablo 4.11: Bit Düzeyi İşleçler

Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
// C#'ta ikili sayıları 0b öneki ile tanımlarız.
byte a = 0b0000_0110; // 6
byte b = 0b0000_0010; // 2

// 1. Bit Düzeyi VE (AND)
// Karşılıklı her iki bit de 1 ise sonuç 1 olur.
byte bitwiseAnd = (byte)(a & b); // 00000010 (2)
// 2. Bit Düzeyi VEYA (OR)
// Karşılıklı bitlerin biri 1 ise sonuç 1 olur.
byte bitwiseOr = (byte)(a | b); // 00000110 (6)
// 3. Bit Düzeyi ÖZEL VEYA (XOR)
// Karşılıklı bitler birbirinden farklı ise sonuç 1 olur.
byte bitwiseXor = (byte)(a ^ b); // 00000100 (4)
// 4. Bit Düzeyi DEĞİL (NOT)
// Tüm bitleri tersine çevirir.
// Not: ~b işlemi sonucu int döneceği için byte'a cast edilir.
byte bitwiseNot = (byte)(~b); // 11111101 (253)
// 5. Sola Kaydırma (Left Shift)
// Bitleri 2 basamak sola kaydırır (Sayıyı 2^2 ile çarpar).
byte leftShift = (byte)(b << 2); // 00001000 (8)
// 6. Sağa Kaydırma (Right Shift)
// Bitleri 1 basamak sağa kaydırır (Sayıyı 2^1'e böler).
byte rightShift = (byte)(b >> 1); // 00000001 (1)
```

İşaretsiz sağa kaydırma (unsigned right shift) işleci (**>>>**), C# 11 ile birlikte dilimize eklenen modern ve oldukça teknik bir işleçtir. Standart sağa kaydırma işlecinin (**>>**) aksine, sayının işaretine (pozitif veya negatif olmasına) bakmaksızın en soldaki boşlukları her zaman 0 ile doldurur. C# dilinde iki tür sağa kaydırma vardır:

1. Aritmetik Kaydırma (**>>**): Sayı negatifse, en soldaki bit (işaret biti) 1 olduğu için boşalan yerleri 1 ile doldurur. Bu, sayının işaretini korur.

2. Mantıksal/İşaretsiz Kaydırma (>>>): Sayı ne olursa olsun, en soldaki boşalan bitleri her zaman 0 ile doldurur.

```
// -8 sayısının ikilik (binary) temsili (32-bit):
// 11111111 11111111 11111111 11111000
int sayi = -8;
// 1. Standart Sağa Kaydırma (>>): İşaret korunur
int aritmetik = sayi >> 2;
// Sonuç: 11111111 11111111 11111111 11111110 (-2)
// 2. İşaretsiz Sağa Kaydırma (>>>): Sol taraf 0 ile dolar
int mantiksal = sayi >>> 2;
// Sonuç: 00111111 11111111 11111111 11111110 (1,073,741,822)
```

§4.10.5 Mantıksal İşleçler

Mantıksal ifadeler, doğru (**true**) ve yanlış (**false**) olabilen ifadelerdir. C# dilinde sıfırdan farklı bir tam sayı ifade **true**, sıfır olan tam sayı yanlış **false** olarak değerlendirilir. **Mantıksal işleçlere** (**logical operator**) işlenen olarak giren ifadeler öncelikle **bool** veri tipine dönüştürülür.

İşleç	Mantıksal İşlem ve Kısa Devre Davranışı
&&	Koşullu Mantıksal VE: Her iki işlenen de true ise true döner. Soldaki işlenen false ise sağdaki ifade değerlendirilmez (Kısa Devre).
 	Koşullu Mantıksal VEYA: İşlenenlerden en az biri true ise true döner. Soldaki işlenen true ise sağdaki ifade değerlendirilmez (Kısa Devre).
!	Mantıksal DEĞİL: İşlenenin mantıksal durumunu tersine çevirir. true değerini false , false değerini true yapar.

Tablo 4.12: Mantıksal İşleçler ve Kısa Devre Özelliği

Bu işleçlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin bir kısmı işlenmediğinden, **koşullu olarak çalışırlar**. Buna **kısa devre** adı verilir. Derleyici, sonucun kesinleştiği noktada işlem yapmayı bırakır. Örneğin (**a == 5 && b == 10**) ifadesinde **a** değeri 5 değilse, **b**'nin ne olduğuna bakmaya gerek duymaz. Bu durum sadece hız kazandırmaz, aynı zamanda olası hataları da önler:

```
int a=10, b=10;
if (a == 5 && b == 10) // a 5 den farklı olduğu için b==10 kontrol edilmez!
{
    // Buradaki işlemler yapılmaz!...
}
```

C# dilinde tekli **&** ve **|** işleçleri de mantıksal ifadelerle kullanılabilir. Ancak bunlar **kısa devre yapmazlar**; yani sol taraf ne olursa olsun sağ tarafı da kontrol ederler. Aşağıdaki kod parçasında **&&** yerine **&** kullanılsaydı programın davranışı nasıl değişirdi?

```
string metin = null;
if (metin != null && metin.Length > 0) // Sorunsuz çalışır.
{
    Console.WriteLine("Metin dolu.");
}
if (metin != null & metin.Length > 0) // PROGRAM ÇÖKER!
```

Verilen kodda **&** işleci kısa devre yapmadığı için **metin, null** olsa bile sağdaki **metin.Length** ifadesini hesaplamaya çalışır ve **NullReferenceException** hatasına sebep olur.

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// 1. Mantıksal VE (&&)
res = (a != 0) && (b != 0); // true
// 2. Mantıksal VEYA (||)
res = (a > 0) || (b > 10); // true
// 3. Mantıksal DEĞİL (!)
bool durum = (a == 5); // true
res = !durum;           // false
```

§4.10.6 Atama işleçleri

Atama işleçleri (**assignment operator**) en çok kullanılan işleçlerdir.

Atama İşleci

Atama işleçleri her zaman sağdaki değeri sola atar! Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (**variable**) olmak zorundadır.

İşleç	Adı	Örnek	Açıklama
=	Temel Atama	x = 5	Sağdaki değeri soldaki değişkene aktarır.
+=	Toplayarak Atama	x += 3	x = x + 3 ile aynıdır.
-=	Çıkararak Atama	x -= 2	x = x - 2 ile aynıdır.
*=	Çarparak Atama	x *= 4	x = x * 4 ile aynıdır.
/=	Bölerek Atama	x /= 2	x = x / 2 ile aynıdır.
%=	Modül Alarak Atama	x %= 3	x = x % 3 ile aynıdır.
&=	Bit Düzeyi VE ile Atama	x &= 1	x = x & 1 ile aynıdır.
=	Bit Düzeyi VEYA ile Atama	x = 1	x = x 1 ile aynıdır.
^=	Bit Düzeyi ÖZEL VEYA ile Atama	x ^= 1	x = x ^ 1 ile aynıdır.
<<=	Bit Düzeyi Sola Kaydırma ile Atama	x <<= 1	x = x << 1 ile aynıdır.
>>=	Bit Düzeyi Sağa Kaydırma ile Atama	x >>= 1	x = x >> 1 ile aynıdır.
>>>=	Bit Düzeyi İşaretsiz Sağa Kaydırma ile Atama	x >>>= 1	x = x >>> 1 ile aynıdır.
??=	Null Atama (C# 8.0+)	x ??= 5	x null ise 5 değerini atar.

Tablo 4.13: Atama İşleçleri

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// 1. Temel Atama
int a = 10; // a değişkenine 10 değişmezi (literal) atanıyor.
// 2. Aritmetik Bileşik Atama İşleçleri
a += 10; // a = a + 10; (a = 20)
a -= 10; // a = a - 10; (a = 10)
a *= 10; // a = a * 10; (a = 100)
a /= 10; // a = a / 10; (a = 10)
a %= 10; // a = a % 10; (a = 0)
// 3. Bit Düzeyi Bileşik Atama İşleçleri
int b = 0b0101; // 5
b &= 0b0011; // b = b & 3; (b = 1 olur)
b |= 0b0010; // b = b | 2; (b = 3 olur)
b ^= 0b0001; // b = b ^ 1; (b = 2 olur)
// 4. Kaydırma Bileşik Atama İşleçleri
int c = 4;
c <<= 1; // c = c << 1; (c = 8)
c >>= 2; // c = c >> 2; (c = 2)
// 5. Null-Coalescing Assignment (C# 8.0+)
string metin = null;
metin ??= "Varsayılan Değer"; // metin null olduğu için "Varsayılan Değer" atandı.
```

§4.10.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C# programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
int a = 10, b = 5;
// 1. Örnek: Çıkarma ve Toplama Aynı Önceliklidir (Soldan Sağa)
a = a - b + 2; // a = 10 - 5 + 2 -> 5 + 2 = 7
// 2. Örnek: Bölme, Toplamadan Önceliklidir
// a = 7 / 5 + 2
// Not: 7/5 tam sayı bölmesi olduğu için 1.4 değil 1 olur.
// a = 1 + 2 = 3
a = a / b + 2; //3
```

Kategori	İşleçler	Birleşme	Öncelik
Temel	<code>x.y, f(x), a[i], x++, x--, new, sizeof</code>	Soldan Sağa	1 (En Yüksek)
Tekli	<code>+x, -x, !x, ~x, ++x, --x, (T)x</code>	Sağdan Sola	2
Çarpma/Bölme	<code>*, /, %</code>	Soldan Sağa	3
Toplama/Çıkarma	<code>+, -</code>	Soldan Sağa	4
Bit Düzeyi Kaydırma	<code><<, >></code>	Soldan Sağa	5
İlişkisel	<code><, >, <=, >=, is, as</code>	Soldan Sağa	6
Eşitlik	<code>==, !=</code>	Soldan Sağa	7
Mantıksal VE	<code>& (Bit Düzeyi/Mantıksal)</code>	Soldan Sağa	8
Mantıksal XOR	<code>^ (Bit Düzeyi/Mantıksal)</code>	Soldan Sağa	9
Mantıksal VEYA	<code> (Bit Düzeyi/Mantıksal)</code>	Soldan Sağa	10
Koşullu VE	<code>&& (Kısa Devre)</code>	Soldan Sağa	11
Koşullu VEYA	<code> (Kısa Devre)</code>	Soldan Sağa	12
Null Birleştirme	<code>??</code>	Sağdan Sola	13
Atama	<code>=, +=, -=, *=, /=, %=, ??=</code>	Sağdan Sola	14 (En Düşük)

Tablo 4.14: İşleç Öncelik Hiyerarşisi

```
// 3. Karmaşık Örnek: Hiyerarşik Çözüm
// Öncelik: ( ) > * , / > + , -
a = 2 * 10 - 20 / 2 + 3 + (12 - 10); //15
/* İfadenin çözüm sırası:
1. Parantez içi: (12 - 10) = 2
2. Çarpma: 2 * 10 = 20
3. Bölme: 20 / 2 = 10
4. Çıkarma (Soldan): 20 - 10 = 10
5. Toplama: 10 + 3 = 13
6. Son Toplama: 13 + 2 = 15
*/
```

4.11 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0.0F;
for(float a = 0.0F; a == 2.0F; a += 0.01F)
    total += a;
Console.WriteLine($"Toplam:{total}"); //Program İcrası Asla Buraya Ulaşmaz!
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199.0 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur:

1. Birincisi yazıldığı haliyle program asla sonuçlanmaz. `a` asla 2'ye eşit olmaz ve döngü asla sonlanmaz.
2. İkincisi ise döngü mantığını `a < 2` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 201'e ulaşır.

Bunun olmasının nedeni, kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```
double a = 0.1;
double b = 0.2;
double c = 0.3;
if(a + b == c)
    Console.WriteLine("Bu satır İcra Edilmez!"); //IEEE754 standardında işlem yaparsak.
else
    Console.WriteLine("Bu satır İcra EDİLİR!"); // Kayan Noktalı Sayılar İkili Sayı Sisteminde
    ↳ Tutulduğundan Yaklaşık Olarak Hesaplanır.
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk

sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki $1/3$ sayısı gibi $0.3333333333333333...$ şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
double a = 0.1;
double b = 0.2;
double c = 0.3;
double toplam = a + b;

// Karşılaştırma sonucu şaşırtıcı olacaktır!
bool esitMi = (toplam == c);
Console.WriteLine($"a (0.1) : {a:R}"); // 'R' formatı tam hassasiyeti gösterir
Console.WriteLine($"b (0.2) : {b:R}");
Console.WriteLine($"a + b : {toplam:R}");
Console.WriteLine($"c (0.3) : {c:R}");
Console.WriteLine($"\\n(0.1 + 0.2) == 0.3 sonucu: " + esitMi);
// Aradaki çok küçük farkı görelim:
double fark = toplam - c;
Console.WriteLine($"Aradaki fark: {fark:R}");

/*Program Çıktısı:
a (0.1) : 0.1
b (0.2) : 0.2
a + b : 0.30000000000000004
c (0.3) : 0.3

(0.1 + 0.2) == 0.3 sonucu: False
Aradaki fark: 5.551115123125783E-17
*/
```

4.12 Düşün ve Çöz

- ♦ **Düşün:** C#'ta değer tipler (**value types**) ile referans tipler (**reference types**) bellek modelinde nasıl farklılaşır? Yığın(**stack**) ve öbek (**heap**) kavramlarını bir araba ve arabanın kopyası metaforuyla açıklayınız.
- ♦ **Düşün:** **int**, **long**, **float** ve **double** arasındaki seçimi etkileyen faktörler nelerdir? Finansal hesaplamalarda **decimal** tipinin tercih edilmesinin teknik nedeni nedir? Ondalıklı sayı hesaplamalarında kayan nokta hatalarına somut bir örnek veriniz.
- ♦ **Düşün:** C#'ta **var** anahtar kelimesiyle tanımlanan değişkenin tipi derleme zamanında mı çalışma zamanında mı belirlenir? **dynamic** ile **var** arasındaki temel fark nedir? Hangi durumda hangisi tercih edilmelidir?
- ♦ **Düşün:** Boş olabilir değer tipleri (**Nullable<T>** veya **int?**) neden C#'a eklenmiştir? Bir veritabanı sütununun 'henüz girilmemiş' değerini temsil etmek için neden düz **int** yeterli değildir?
- ♦ **Düşün:** C#'taki işleç önceliği (**operator precedence**) kuralları bilinmediğinde nasıl beklenmedik sonuçlar çıkabilir? Parantez kullanmanın hem doğruluk hem de okunabilirlik açısından önemi nedir?
- ♦ **Düşün:** Numaralandırma tipi (**enum**) yerine **string** sabitleri kullanmanın yaratacağı sorunları tartışınız. Enum kullanımı kodun bakım kolaylığını nasıl artırır?
- ♦ **Düşün:** C#9+ ile gelen kayıt tipleri (**record**) ve değiştirilemez nesneler (**immutable objects**) kavramı neden modern yazılım mimarisinde önem kazanmaktadır? İş parçacığı-güvenli (**thread-safe**) olması açısından faydası nedir?
- ♦ **Çöz:** **int**, **double**, **bool**, **char** ve **string** tiplerinden birer değişken tanımlayınız. Her birinin bellekte kapladığı alanı belirtiniz ve her birine uygun bir gerçek hayat senaryosu yazınız.
- ♦ **Çöz:** Aşağıdaki ifadelerin C#'ta ürettiği değerleri hesaplayınız:
a) $17 \% 5$ b) $(int)(7.9)$ c) $2 \ll 3$ d) $0xFF \& 0x0F$ e) $true \&\& (5 > 3)$
- ♦ **Çöz:** Bir dikdörtgenin alanını ve çevresini hesaplayan, enini ve boyunu kullanıcıdan alan C# programını yazınız. Girilen değerlerin sıfırdan büyük olup olmadığını kontrol ediniz.
- ♦ **Çöz:** Bir sıcaklık değerini Celsius'tan Fahrenheit'a ve Kelvin'e dönüştüren programı yazınız. Mutlak sıfır (-273.15°C) altında giriş yapılmasını engelleyiniz.
- ♦ **Çöz:** C#'ta **int.MaxValue** değerine 1 eklendiğinde ne olur? Taşma (**overflow**) durumunu **checked** anahtar kelimesiyle nasıl yakalarsınız? Deneyi yapıp sonucu açıklayınız.
- ♦ **Düşün:** Neden **if (0.1 + 0.2 == 0.3)** ifadesi **false** sonucunu verir? Bu durumun donanımsal sebebi nedir?
- ♦ **Düşün:** Değişken isimlendirirken neden Türkçe karakter kullanabiliyoruz ama anahtar kelimeler (**if**, **while** vb.) sadece İngilizce karakterlerden oluşmak zorundadır?

- ◇ **Düşün:** UTF-8 karakter seti desteği, kaynak kod yazarken bize ne gibi avantajlar sağlar?